class06: R projects

Kate Ruiz (PID: A17671200)

Table of contents

Background

Functions are at the heart of using R. Everything we do involves calling and using functions (from data input, analysis to results output).

All functions in R have at least 3 things:

1. A **name** the thing we use to call the function.
2. One or more input **arguments** that are comma separated
3. The **body**, lines of code between curly brackets {} that does the work of the function.

A first function

Let's write a silly wee function to add some numbers:

```
add <- function(x) {
  x + 1
}
```

Let's try it out

```
add(100)
```

```
[1] 101
```

Will this work

```
add(c(100,200,300))
```

```
[1] 101 201 301
```

Modify to be more useful and add more than just 1

```
add <- function(x, y=1) {
  x + y
}
```

```
add(100, 10)
```

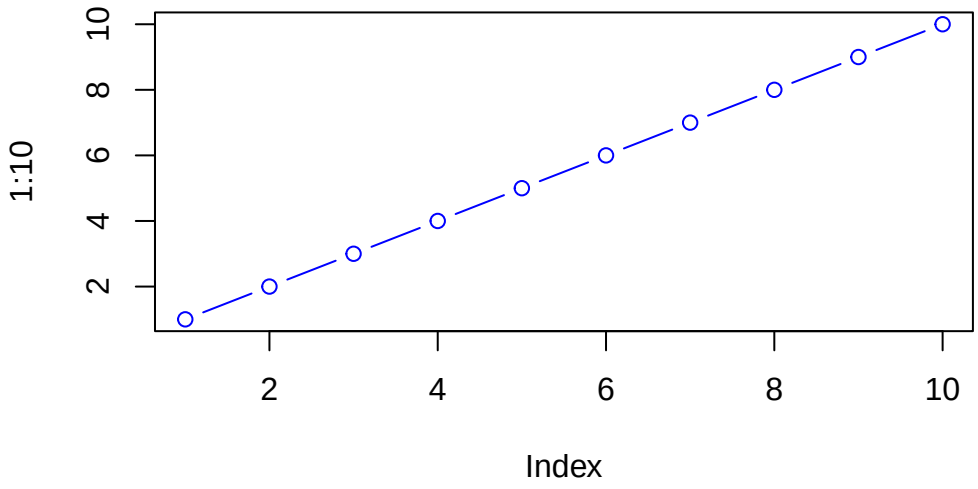
```
[1] 110
```

Will this work?

```
add(100,0)
```

```
[1] 100
```

```
plot(1:10, col="blue", typ="b")
```




```
log(10, base=10)
```

```
[1] 1
```

> **Note to Kate** Input arguments can be either **required** or **optional**. The later have a fall-back default that is specified in the function code with an equals sign.

```
#add(x=100, y=200, z=300)
```

A second function

all functions in R look like this

```
name <- function(arg) {
  body
}
```

The `sample()` function in R ...

```
sample(1:10, size=4)
```

```
[1] 4 6 5 8
```

Q. Return 12 numbers picked randomly from the input 1:10

```
sample(1:10, size=12, replace=TRUE)
```

```
 [1]  7 10  2  6 10  3  1  5 10  5  5  9
```

Q. Write the code to generate a random 12 nucleotide long DNA sequence?

```
nucleotide <- c("A", "T", "G", "C")
sample(nucleotide, size=12, replace=TRUE)
```

```
 [1] "G" "G" "G" "G" "G" "T" "C" "A" "G" "G" "G" "T"
```

Q. Write a first version function called `generate_dna()` that generates a user specified length `n` random DNA sequence?

```
name <- function(arg) {
  body
}
```

```
generate_dna <-function(n=6){
  nucleotide <- c("A", "T", "G", "C")
  sample(nucleotide, size=n, replace=TRUE)
}
```

```
generate_dna(100)
```

```
  [1] "T" "T" "A" "G" "G" "C" "G" "G" "A" "C" "C" "A" "A" "T" "C" "C" "G" "G"
 [19] "A" "T" "T" "A" "T" "A" "T" "A" "T" "T" "G" "C" "A" "G" "T" "T" "T" "A"
 [37] "A" "C" "T" "T" "G" "G" "C" "G" "C" "T" "A" "T" "G" "G" "T" "T" "C" "A"
 [55] "C" "G" "C" "A" "T" "T" "G" "C" "C" "A" "C" "A" "T" "C" "C" "A" "A" "C"
 [73] "G" "T" "T" "T" "G" "C" "T" "G" "G" "T" "A" "G" "C" "G" "C" "G" "A" "C"
 [91] "T" "T" "A" "A" "G" "T" "G" "C" "A" "C"
```

Q. Modify your function to return a FASTA like sequence so rather than [1] “G” “T” “T” “G” “T” “C” “G” “A” “G” “G” “A” “G” we want “GTTG…”

```
generate_dna <-function(n=6){
  nucleotide <- c("A", "T", "G", "C")
  sally<-sample(nucleotide, size=n, replace=TRUE)
  sally <- paste(sally, collapse="")
  return(sally)
}
```

by default the last thing you type is what you get

```
generate_dna(10)
```

```
[1] "CCTAGTATGC"
```

> Q *bingus is a hairless cat* Give the user an option to return FASTA format output ssequence or standard multi-element vector format

```
generate_dna <-function(n=6, fasta=TRUE){
  nucleotide <- c("A", "T", "G", "C")
  sally<-sample(nucleotide, size=n, replace=TRUE)

  if(fasta){
    sally <- paste(sally, collapse="")
    cat("Hellloooooooo")

  }
  return(sally)
}
```

```
generate_dna(10, fasta=T)
```

```
Hellloooooooo
```

```
[1] "CGCCATGGCA"
```

A new col function

Q. Write a function called `generate_protein()` that generates a user specified length protein sequence in FASTA like format?

Q. Use your new `generate_protein()` function to generate sequences between length 6 and 12 amino acids in length and check if any of these are unique in nature (i.e. found in the NR database at NCBI)

```
generate_protein <- function(n, fasta=TRUE){
  amino_acids <- c("A","R","N","D","C",
  "E","Q","G","H","I",
  "L","K","M","F","P",
  "S","T","W","Y","V")
  more<-sample(amino_acids, size=n, replace=TRUE)
  if(fasta){
    more<-paste(more, collapse ="")
  }
  return(more)
}
```

```
generate_protein(6, T)
```

```
[1] "RMAMLQ"
```

```
generate_protein(7, T)
```

```
[1] "TNHPAYF"
```

```
generate_protein(8, T)
```

```
[1] "LDNVHQCT"
```

```
generate_protein(9, T)
```

```
[1] "SCMTPPNMG"
```

```
generate_protein(10, T)
```

```
[1] "IAECAYMWIN"
```

```
generate_protein(11, T)
```

```
[1] "PYWFQFPFYVW"
```

```
generate_protein(12, T)
```

```
[1] "SSQLLTDIRSKC"
```

Or we could do a `for()` loop:

```
for (i in 6:12) {
  cat(i, "\n")
}
```

```
6
7
8
9
10
11
12
```

```
for (i in 6:12) {
  cat(">", i,sep="", "\n")
  cat(generate_protein(i), "\n")
}
```

```
>6
ERGGAM
>7
AFNFKGL
>8
SLRGFAVV
>9
```

```
FNGKHDHTM
>10
QLSSNTRWDL
>11
VVQKGMILRAF
>12
HCPEGFNACVVS
```

id AGKRT